

EIE3360 Integrated Project

Lecture 1 In-class questions

Q3: Image Classification of Handwritten Digits

Objective:

Students will gather image data, preprocess it, train a convolutional neural network (CNN) to classify handwritten digits, and evaluate its performance.

1. Data Gathering

Dataset:

- Use the MNIST dataset, which is a well-known dataset of handwritten digits. It contains 60,000 training images and 10,000 testing images, each of size 28x28 pixels.

Download the Dataset:

- The MNIST dataset can be easily accessed through libraries like TensorFlow or Keras.

```
import tensorflow as tf

# Load the MNIST dataset
mnist = tf.keras.datasets.mnist
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

2. Data Preprocessing

Normalize the Data:

- Normalize the pixel values to be between 0 and 1 by dividing by 255.

Reshape the Data:

- Reshape the data to add a channel dimension, as required by CNNs.

```
# Normalize the data
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape the data
X_train = X_train.reshape(-1, 28, 28, 1)
```

```
X_test = X_test.reshape(-1, 28, 28, 1)
```

3. Model Training

Build the CNN Model:

- Use Keras to build a simple CNN model.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Input

# Build the CNN model
model = Sequential([
    Input(shape=(28, 28, 1)), # Define the input shape using the Input layer
    Conv2D(32, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

Train the Model:

- Train the model using the training data.

```
# Train the model
model.fit(X_train, y_train, epochs=5, validation_data=(X_test, y_test))
```

4. Model Evaluation

Evaluate the Model:

- Evaluate the model on the test data.

```
from tensorflow.keras import backend as K
from sklearn.metrics import classification_report
```

```
# Generate predictions
y_pred_prob = model.predict(X_test)
y_pred = y_pred_prob.argmax(axis=1) # Get class predictions

# Classification report for precision, recall, and F1-score
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

Confusion Matrix:

- Generate a confusion matrix to visualize the model's performance.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
import seaborn as sns

# Predict the labels for the test set
y_pred = np.argmax(model.predict(X_test), axis=-1)

# Generate the confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)

# Plot the confusion matrix
plt.figure(figsize=(10, 8))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```

Show the 10x10 confusion matrix to our TAs/Technicians.

Upload your .ipynb file and confusion matrix to Blackboard.